Web Development – Exam 2 (WEB26-1-14, 2026-01-14) – with answers

The exam is composed of short answer (max. 50 words), code comprehension, and multiple choice questions. Duration: two hours. Max 50 points.
3-5 points for each code comprehension question, 1 point for each multiple-choice question.

Grade	Points
U	< 20
3	20-29
4	30-39
5	40-50

How to read this file: the green ● option is the correct answer. The grey text in parentheses is the Swedish translation.
(Så läser du filen: det gröna ●-alternativet är rätt svar. Den grå texten inom parentes är svensk översättning.)

---

1 T1_General_Q1_20260114

Which statement best describes the client–server model?
(Vilket påstående beskriver bäst klient–server-modellen?)

Select one alternative:

- ○ The server always initiates communication.
- ○ Both client and server execute the same code.
- ● **The client requests services, and the server responds. ✓**
- ○ The server stores the client's source code.

Answer:
The client (the browser) asks, and the server answers. The server does not start the talk by itself.
(Klienten (webbläsaren) frågar och servern svarar. Servern startar inte kommunikationen själv.)

Totalpoäng: 1

---

2 T1_General_Q2_20260114

What is a static website?
(Vad är en statisk webbsida?)

Select one alternative:

- ● **A site where content is fixed and served as-is from the server ✓**
- ○ One that changes content based on user interaction
- ○ A site that requires authentication
- ○ A site that uses APIs for content updates

Answer:
The same files are sent to everyone. Nothing is created or changed per user.
(Samma filer skickas till alla. Inget skapas eller ändras per användare.)

Totalpoäng: 1

3 T1_General_Q3_20260114

Ersätt med din uppgiftstext... (the teacher forgot to write the question)
(Läraren har glömt skriva frågan – det finns inget att svara på.)

Select one alternative:

- ○ Alternativ 3
- ○ Alternativ 1
- ○ Alternativ 2
- ○ Alternativ 4

Answer:
No real question here. Skip it.
(Ingen riktig fråga här. Hoppa över den.)

Totalpoäng: 1

4 T2_HTML/CSS/JS_Q1_20260114

What does DOM stand for?
(Vad står DOM för?)

Answer:
Document Object Model. It is the tree of objects that represents the HTML page. JavaScript uses it to read and change the page. (The options were not visible in the PDF.)
(Document Object Model. Det är trädet av objekt som representerar HTML-sidan. JavaScript använder det för att läsa och ändra sidan. (Alternativen syntes inte i PDF:en.))

Totalpoäng: 1

5 T2_HTML/CSS/JS_Q2_20260114

Which CSS selector targets all <p> elements inside a ?
(Vilken CSS-selektor väljer alla <p>-element inuti en ?)

Select one alternative:

- ○ div+p
- ○ p div
- ○ div>p
- ● **div p ✔**

Answer:
A space means ALL p inside the div, at any depth. div>p = only direct children. div+p = the p right AFTER the div.
(Mellanslag betyder ALLA p inuti div, hur djupt som helst. div>p = bara direkta barn. div+p = p:n direkt EFTER div.)

Totalpoäng: 1

6 T2_HTML/CSS/JS_Q3_20260114

What will be logged to the console?
(Vad loggas i konsolen?)

```
let text = "Hello";
text[0] = "J";
console.log(text);
```

Select one alternative:

- ● **Hello ✔**
- ○ undefined
- ○ Error
- ○ Jello

Answer:
Strings cannot be changed (they are immutable). text[0] = "J" is just ignored. (In strict mode it gives an error, but the expected answer is Hello.)

(Strängar går inte att ändra (de är immutable). text[0] = "J" ignoreras bara. (I strict mode blir det fel, men svaret de vill ha är Hello.))

Totalpoäng: 1

7 T2_HTML/CSS/JS_Q4_20260114

Which HTML5 element is used to represent self-contained content such as a blog post or article?
(Vilket HTML5-element används för fristående innehåll, som ett blogginlägg eller en artikel?)

Select one alternative:

- ● **<article> ✔**
- ○ <section>
- ○
- ○ <content>

Answer:
<article> = content that can stand alone. <section> = a part of a page. has no meaning. <content> does not exist.

(<article> = innehåll som kan stå själv. <section> = en del av en sida. har ingen betydelse. <content> finns inte.)

Totalpoäng: 1

8 T2_HTML/CSS/JS_Q5_20260114

What is logged to the console?
(Vad loggas i konsolen?)

```
var x = 5;
function test() {
  console.log(x);
  var x = 10;
}
test();
```

Select one alternative:

- ○ 5
- ● **undefined ✔**
- ○ 10
- ○ error

Answer:
Hoisting! The local "var x" is moved to the top of the function and hides the global x = 5. The local x is still undefined when console.log runs.

(Hoisting! Det lokala "var x" flyttas upp i funktionen och döljer det globala x = 5. Det lokala x är fortfarande undefined när console.log körs.)

Totalpoäng: 1

9 T3_Front-end Programming/React_Q1_20260114

What is JSX?
(Vad är JSX?)

Select one alternative:

- ○ A database query language
- ○ A CSS framework
- ○ A React state hook
- ● **HTML syntax embedded in JavaScript ✔**

Answer:
JSX looks like HTML but is written inside JavaScript. It is turned into normal JS calls.

(JSX ser ut som HTML men skrivs inuti JavaScript. Den görs om till vanliga JS-anrop.)

Totalpoäng: 1

10 T3_Front-end Programming/React_Q2_20260114

Which hook is used to add state to a functional component?
(Vilken hook används för att lägga till state i en funktionskomponent?)

Select one alternative:

- ○ useCallback
- ● **useState ✔**
- ○ useRef
- ○ useEffect

Answer:
useState gives you [value, setValue].
(useState ger dig [värde, setVärde].)

Totalpoäng: 1

11 T3_Front-end Programming/React_Q3_20260114

When does a useEffect hook run if you provide an empty dependency array []?
(När körs useEffect om man ger en tom dependency-array []?)

Select one alternative:

- ● **Only once after the first render ✔**
- ○ It never runs
- ○ Only when state changes
- ○ On every render

Answer:
Empty array = nothing it depends on can change, so it runs only once, after the first render.
(Tom array = inget den beror på kan ändras, så den körs bara en gång, efter första renderingen.)

Totalpoäng: 1

12 T3_Front-end Programming/React_Q4_20260114

What will this output?
(Vad blir resultatet?)

```
const App = () => {
  const [count, setCount] = React.useState(0);
  setCount(1);
  console.log(count);
  return <div>{count}</div>;
};
```

Select one alternative:

- ○ 1
- ○ nothing
- ● **error ✔**
- ○ 0

Answer:
Calling setCount directly in the component body starts a new render, which calls setCount again, again... React stops it with the error "Too many re-renders". Put setState in an event handler or useEffect instead.

(Att anropa setCount direkt i komponentens kropp startar en ny rendering, som anropar setCount igen, och igen... React stoppar det med felet "Too many re-renders". Lägg setState i en event handler eller useEffect istället.)

Totalpoäng: 1

13 T3_Front-end Programming/React_Q5_20260114

What is the output of the following script? Please also provide a short explanation.
(Vad blir utskriften? Ge också en kort förklaring.)

```
const App = () => {
  const [count, setCount] = React.useState(0);
  React.useEffect(() => {
    setCount(5);
  }, []);
  console.log(count);
  return <div>{count}</div>;
};
```

Answer:
Output: 0, then 5.
First render: count is 0, so it logs 0. After the render, the effect runs once and sets count to 5. That causes a new render, which logs 5. The effect does not run again because of [].

(Utskrift: 0, sen 5. Första renderingen: count är 0, så den loggar 0. Efter renderingen körs effekten en gång och sätter count till 5. Det ger en ny rendering, som loggar 5. Effekten körs inte igen på grund av [].)

Totalpoäng: 1

14 T4_BackEnd/Node.js_Q1_20260114

What is Express.js?
(Vad är Express.js?)

Select one alternative:

- ○ A CSS framework
- ● **A Node.js framework for building web applications and APIs ✔**
- ○ A front-end library
- ○ A database query language

Answer:
Express makes it easy to build routes and middleware on top of Node.
(Express gör det enkelt att bygga routes och middleware ovanpå Node.)

Totalpoäng: 1

15 T4_BackEnd/Node.js_Q2_20260114

What does `require()` do in Node.js?
(Vad gör `require()` i Node.js?)

Select one alternative:

- ● **Imports modules into a file ✔**
- ○ Compiles JavaScript
- ○ Sends HTTP requests
- ○ Creates a new server

Answer:
const fs = require('fs') brings a module into your file so you can use it.
(const fs = require('fs') hämtar in en modul i din fil så du kan använda den.)

Totalpoäng: 1

16 T4_BackEnd/Node.js_Q3_20260114

What does `app.use()` do in Express?
(Vad gör `app.use()` i Express?)

Select one alternative:

- ○ Parses JSON
- ○ Starts the server
- ○ Defines a GET route
- ● **Registers middleware ✔**

Answer:
app.use adds middleware. app.get = GET route, app.listen = start the server.
(app.use lägger till middleware. app.get = GET-route, app.listen = starta servern.)

Totalpoäng: 1

17 T4_BackEnd/Node.js_Q4_20260114

What will this code output in the console?
(Vad skriver koden ut i konsolen?)

```
const fs = require('fs');
console.log('Start');
fs.readFile('file.txt', 'utf8', (err, data) => {
  console.log('File read');
});
console.log('End');
```

Select one alternative:

- ○ File read / Start / End
- ○ File read
- ○ Start / File read / End
- ● **Start / End / File read ✔**

Answer:
readFile is asynchronous. It reads the file in the background, and the callback runs only when it is done, after the normal code.

(readFile är asynkron. Den läser filen i bakgrunden, och callbacken körs först när den är klar, efter den vanliga koden.)

Totalpoäng: 1

18 T4_BackEnd/Node.js_Q5_20260114

Which method is used to start an Express server?
(Vilken metod används för att starta en Express-server?)

Select one alternative:

- ○ server.run()
- ○ app.start()
- ○ express.begin()
- ● **app.listen() ✔**

Answer:
app.listen(3000) makes the server listen on port 3000.

(app.listen(3000) gör att servern lyssnar på port 3000.)

Totalpoäng: 1

19 T5_Browser/server_communication_Q1_20260114

What does a 404 HTTP status code indicate?
(Vad betyder HTTP-statuskoden 404?)

Select one alternative:

- ● **Resource not found ✔**
- ○ Server error
- ○ Unauthorized access
- ○ Request successful

Answer:
4xx = the client's error. 404 = the thing you asked for does not exist.
(4xx = klientens fel. 404 = det du bad om finns inte.)

Totalpoäng: 1

20 T5_Browser/server_communication_Q2_20260114

Which tool is commonly used to test HTTP APIs?
(Vilket verktyg används ofta för att testa HTTP-API:er?)

Select one alternative:

- ● **Postman ✔**
- ○ Babel
- ○ Nginx
- ○ Docker

Answer:
Postman sends requests and shows the answers. Babel = compiler, Nginx = web server, Docker = containers.
(Postman skickar requests och visar svaren. Babel = kompilator, Nginx = webbserver, Docker = containrar.)

Totalpoäng: 1

21 T5_Browser/server_communication_Q3_20260114

What is the main purpose of CORS (Cross-Origin Resource Sharing)?
(Vad är huvudsyftet med CORS?)

Select one alternative:

- ○ To encrypt HTTP data
- ○ To cache responses locally
- ● **To allow browsers to access APIs hosted on different domains ✔**
- ○ To reduce server load

Answer:
The browser normally blocks requests to other domains. CORS headers from the server allow it.
(Webbläsaren blockerar normalt anrop till andra domäner. CORS-headers från servern tillåter det.)

Totalpoäng: 1

22 T5_Browser/server_communication_Q4_20260114

What will this code output if the server returns a 500 Internal Server Error with a JSON response body?

(Vad skriver koden ut om servern svarar 500 Internal Server Error med en JSON-body?)

```
fetch('https://api.example.com/data')
  .then(res => res.json())
  .then(data => console.log('Data:', data))
  .catch(err => console.log('Error:', err));
```

Select one alternative:

- ● **Logs "Data:" with parsed JSON content ✔**
- ○ Throws an exception automatically
- ○ Crashes the program
- ○ Logs "Error: …" because .catch() handles non-200 responses

Answer:

TRAP: fetch does NOT fail on 404 or 500. It only goes to .catch on network errors. The response counts as OK, the JSON is parsed and logged. That's why you should check res.ok yourself.

(FÄLLA: fetch misslyckas INTE vid 404 eller 500. Den går bara till .catch vid nätverksfel. Svaret räknas som okej, JSON:en parsas och loggas. Därför ska man själv kolla res.ok.)

Totalpoäng: 1

23 T6_Database_Q1_20260114

Which of the following is true about browser cookies?

(Vad stämmer om cookies i webbläsaren?)

Select one alternative:

- ○ They are stored on the client only
- ○ They are deleted automatically after 5 minutes
- ○ They are encrypted by default
- ● **They can be accessed and modified by both client and server ✔**

Answer:

The server sets them (Set-Cookie) and reads them on every request. JavaScript can read/change them with document.cookie (if not HttpOnly). They are not encrypted by default.

(Servern sätter dem (Set-Cookie) och läser dem vid varje request. JavaScript kan läsa/ändra dem via document.cookie (om de inte är HttpOnly). De är inte krypterade som standard.)

Totalpoäng: 1

24 T6_Database_Q2_20260114

In MongoDB, which structure holds documents?
(Vilken struktur i MongoDB innehåller dokument?)

Select one alternative:

- ○ Schema
- ○ Database
- ○ Table
- ● **Collection ✔**

Answer:
Database → Collection → Document. A collection is like a table in SQL.
(Databas → Collection → Dokument. En collection är ungefär som en tabell i SQL.)

Totalpoäng: 1

25 T6_Database_Q3_20260114

What is the purpose of _id in MongoDB documents?
(Vad är syftet med _id i MongoDB-dokument?)

Select one alternative:

- ○ To store timestamps
- ○ To hold metadata for indexing
- ● **To uniquely identify each document ✔**
- ○ To represent relationships between collections

Answer:
_id is unique for every document, like a primary key. It is created automatically.
(_id är unik för varje dokument, som en primärnyckel. Den skapas automatiskt.)

Totalpoäng: 1

26 T7_Security_Q1_20260114

Which of the following is an example of spoofing?
(Vilket är ett exempel på spoofing?)

Select one alternative:

- ○ A server experiences hardware failure
- ○ A database is deleted accidentally
- ○ An attacker floods the server with traffic
- ● **An attacker gains unauthorized access by pretending to be a legitimate user ✔**

Answer:
Spoofing = pretending to be someone else. Flooding the server is DoS.
(Spoofing = att låtsas vara någon annan. Att översvämma servern är DoS.)

Totalpoäng: 1

27 T7_Security_Q2_20260114

Which threat is an example of "Information Disclosure"?
(Vilket hot är ett exempel på "Information Disclosure" (informationsläcka)?)

Select one alternative:

- ○ A user account is locked after too many failed logins
- ○ A database is unavailable due to heavy load
- ○ A malicious script changes the site's homepage
- ● **Sensitive user data is sent over an unencrypted channel ✔**

Answer:
Information disclosure = secret data leaks to someone who should not see it. Changed homepage = Tampering. Unavailable = DoS.
(Information disclosure = hemlig data läcker till någon som inte ska se den. Ändrad hemsida = Tampering. Otillgänglig = DoS.)

Totalpoäng: 1

28 T8_Performance_Q1_20260114

What is caching primarily used for?
(Vad används caching främst till?)

Select one alternative:

- ○ Compressing files
- ○ Preventing SQL injection
- ● **Storing temporary data to improve performance ✔**
- ○ Improving security

Answer:
Save something you already fetched, so you don't have to fetch or compute it again.
(Spara något du redan hämtat, så du slipper hämta eller räkna ut det igen.)

Totalpoäng: 1

29 T8_Performance_Q2_20260114

What does load balancing do?
(Vad gör lastbalansering?)

Select one alternative:

- ○ Compresses responses
- ● **Distributes incoming traffic across multiple servers ✔**
- ○ Stores cache data
- ○ Encrypts traffic

Answer:
The load balancer spreads the traffic so no single server gets overloaded.
(Lastbalanseraren sprider trafiken så att ingen enskild server blir överbelastad.)

Totalpoäng: 1

30 T9_StateManagement_Q1_20260114

What problem does the Context API primarily solve?
(Vilket problem löser Context API främst?)

Select one alternative:

- ● **Prop drilling in deeply nested components ✓**
- ○ Routing between pages
- ○ API communication
- ○ CSS styling issues

Answer:
Prop drilling = passing props through many levels that don't use them. Context skips that.
(Prop drilling = att skicka props genom många nivåer som inte använder dem. Context hoppar över det.)

Totalpoäng: 1

31 T9_StateManagement_Q2_20260114

Which Context API function provides the global state to the component tree?
(Vilken del av Context API ger det globala state:t till komponentträdet?)

Select one alternative:

- ● **Context.Provider ✓**
- ○ Context.Consumer
- ○ useState
- ○ ReactDOM.render

Answer:
<MyContext.Provider value={...}> wraps the components that should get the value.
(<MyContext.Provider value={...}> omsluter komponenterna som ska få värdet.)

Totalpoäng: 1

32 T10_CodeComprehension_Q1_20260114

What will the following script print? Please also provide a short explanation.
(Vad skriver koden ut? Ge också en kort förklaring.)

```
console.log("5" + 1);
```

Answer:
It prints 51 (the string "51").
When + has a string on one side, it joins the values as text. The number 1 becomes "1" and is added to the end of "5". (But "5" - 1 gives 4, because minus only works with numbers.)
(Den skriver ut 51 (strängen "51"). När + har en sträng på ena sidan slås värdena ihop som text. Talet 1 blir "1" och läggs på efter "5". (Men "5" - 1 ger 4, eftersom minus bara fungerar med tal.))

Totalpoäng: 3

33 T10_CodeComprehension_Q2_20260114

What happens when this component renders? Please also provide a short explanation.
(Vad händer när komponenten renderas? Ge också en kort förklaring.)

```
function App() {
  const [count, setCount] = React.useState(0);
  setCount(count + 1);
  return <div>{count}</div>;
}
```

Answer:
Infinite render loop. React crashes with the error "Too many re-renders. React limits the number of renders to prevent an infinite loop." Nothing is shown.
setCount is called directly in the component body. It changes state → new render → setCount again → forever. Fix: call setCount in an event handler (onClick) or in useEffect.

(Oändlig render-loop. React kraschar med felet "Too many re-renders". Inget visas. setCount anropas direkt i komponentens kropp. Det ändrar state → ny rendering → setCount igen → för alltid. Lösning: anropa setCount i en event handler (onClick) eller i useEffect.)

Totalpoäng: 5

34 T10_CodeComprehension_Q3_20260114

What does this render? Please also provide a short explanation.
(Vad renderas? Ge också en kort förklaring.)

```
function App() {
  const [name, setName] = React.useState("Bob");
  return <p>Hello {name}</p>;
}
```

Answer:
It renders Hello Bob (the text "Hello Bob").
useState("Bob") sets name to "Bob". {name} in JSX prints the value. setName is never called, so it stays "Bob".

(Den renderar Hello Bob (texten "Hello Bob"). useState("Bob") sätter name till "Bob". {name} i JSX skriver ut värdet. setName anropas aldrig, så det förblir "Bob".)

Totalpoäng: 5

35 T10_CodeComprehension_Q4_20260114

Explain why the count doesn't reset when clicking "Increment."

(Förklara varför count inte nollställs när man klickar på "Increment".)

```
function Counter() {
  const [count, setCount] = React.useState(0);
  const increment = () => setCount(prev => prev + 1);
  return (
    <div>
      <p>{count}</p>
      <button onClick={increment}>Increment</button>
    </div>
  );
}
```

Answer:

React keeps state between renders. useState(0) uses 0 only on the FIRST render. On later renders React gives back the saved value. A click calls setCount(prev => prev + 1), which takes the old value and adds 1. The component re-renders with the new value. So running the function again does not reset count. The updater function (prev => prev + 1) also makes sure it always counts from the latest value.

(React sparar state mellan renderingar. useState(0) använder 0 bara vid FÖRSTA renderingen. Vid senare renderingar ger React tillbaka det sparade värdet. Ett klick anropar setCount(prev => prev + 1), som tar det gamla värdet och lägger till 1. Komponenten renderas om med det nya värdet. Att funktionen körs igen nollställer alltså inte count. Updater-funktionen (prev => prev + 1) ser också till att den alltid räknar från senaste värdet.)

Totalpoäng: 6